

Week-6 Assignment

Spark Architecture and Efficient Data Processing

Q1: Explain the roles of the Driver, Cluster Manager, and Executor in a Spark application.

In a Spark application, the Driver is the main program that controls the execution. It creates the SparkSession or SparkContext, divides the job into smaller tasks, and sends those tasks for execution.

The Cluster Manager is responsible for managing resources in the cluster. It decides where the executors will run and how much CPU or memory will be given to them. Examples of cluster managers are YARN, Standalone, Mesos, and Kubernetes.

The Executors are worker processes that actually run the tasks assigned by the driver. They process the data, store intermediate results, and send the final result back to the driver.

So in simple words, the Driver plans the work, the Cluster Manager gives resources, and Executors do the actual processing.

Q2: How does Spark's Lazy Evaluation strategy improve performance when chain-processing large datasets?

Spark uses Lazy Evaluation, which means it does not execute transformations immediately. When we write operations like `filter()`, `select()`, or `withColumn()`, Spark only records them in a plan.

The actual execution starts only when an action like `show()`, `count()`, or `write()` is called.

This improves performance because Spark gets time to optimize the full chain of operations. It can remove unnecessary steps, combine transformations, and reduce the amount of data processed. For large datasets, this is very useful because Spark avoids doing extra work.

Example:

```
df2 = df.filter(df["amount"] > 1000).select("order_id", "amount")  
df2.show()
```

Here, Spark will not run the filter and select separately. It will prepare an optimized execution plan and run it only when show() is called.

Q3: Write a Spark command to read a CSV file located at "data/source.csv", ensuring the first row is treated as a header and inferSchema is enabled.

```
df = spark.read.csv(  
    "data/source.csv",  
    header=True,  
    inferSchema=True  
)
```

```
df.show(5)
```

```
df.printSchema()
```

Q4: What is the difference between CSV and Parquet in terms of storage and why does it matter for performance?

CSV is a row-based file format. It stores data row by row in plain text form. It is easy to read and open, but it is slower for big data processing because Spark may need to scan more data.

Parquet is a columnar file format. It stores data column by column. This is better for analytics because Spark can read only the required columns instead of reading the full dataset.

For example, if we only need product_id and price, Parquet can load only these columns. This reduces memory usage and improves performance.

So, CSV is simple and readable, while Parquet is better for large datasets and faster processing.

Q5: Given a DataFrame df, write a query to select the columns product_id and price where the category is 'Electronics'.

```
electronics_df = df.filter(df["category"] == "Electronics") \
    .select("product_id", "price")
```

```
electronics_df.show()
```

Q6: Write the code to revise a DataFrame by renaming the column old_name to new_name and casting the price column from String to Double.

```
from pyspark.sql.functions import col
```

```
revised_df = df.withColumnRenamed("old_name", "new_name") \
    .withColumn("price", col("price").cast("double"))
```

```
revised_df.printSchema()
```

```
revised_df.show(5)
```

Q7: How does Spark use the Lineage Graph/DAG to provide fault tolerance if a worker node fails?

Spark keeps track of all transformations using a Lineage Graph, also called a DAG. DAG means Directed Acyclic Graph.

Whenever we apply transformations on a DataFrame, Spark remembers the steps used to create it. If a worker node fails, Spark does not need to start the whole job again from the beginning. It checks the lineage and recomputes only the lost partition using the same transformation steps.

This makes Spark fault tolerant. It can recover lost data automatically without needing manual work.

Example:

```
filtered_df = df.filter(df["amount"] > 1000)
```

Spark remembers that `filtered_df` was created by applying a filter on `df`. If one partition is lost, Spark can recreate only that partition using the lineage information.

Q8: Write a query to filter a DataFrame `df_orders` for rows where the status is 'Completed' AND the amount is greater than 1000.

```
completed_orders = df_orders.filter(  
    (df_orders["status"] == "Completed") & (df_orders["amount"] > 1000)  
)
```

```
completed_orders.show()
```

Q9: Explain the concept of Predicate Pushdown in Parquet and how it affects the amount of data loaded into memory.

Predicate Pushdown means Spark pushes the filter condition closer to the data source before loading the full data into memory.

In Parquet, data is stored in columnar format with metadata. Because of this, Spark can check conditions before reading unnecessary data.

For example:

```
df = spark.read.parquet("sales.parquet")  
filtered_df = df.filter(df["region"] == "North")
```

Here, Spark can use Predicate Pushdown to read only the required data blocks where region is North. It does not need to load the full file into memory.

This improves performance because less data is read, memory usage is reduced, and processing becomes faster.

Q10: Write a code snippet to add a new column `final_price` which is the `base_price` multiplied by 1.18.

```
from pyspark.sql.functions import col
```

```
df_with_final_price = df.withColumn(  
    "final_price",  
    col("base_price") * 1.18  
)
```

```
df_with_final_price.show()
```

Q11: What is the difference between Transformations and Actions? Provide two examples of each.

Transformations are operations that create a new DataFrame from an existing DataFrame. They are lazy, which means they do not run immediately.

Examples of transformations:

```
df.filter(df["amount"] > 1000)
```

```
df.select("product_id", "price")
```

Actions are operations that trigger actual execution in Spark. When an action is called, Spark starts processing the data.

Examples of actions:

```
df.show()
```

```
df.count()
```

So, transformations prepare the plan, and actions execute the plan.

Q12: Write the Spark command to load a Parquet file from "path/to/input", filter out any rows where user_id is null, and save the result as a CSV at "path/to/output".

```
filtered_users = spark.read.parquet("path/to/input") \  
    .filter("user_id IS NOT NULL")
```

```
filtered_users.write.mode("overwrite") \
    .option("header", True) \
    .csv("path/to/output")
```

Q13: In Spark Architecture, what is the difference between Client Mode and Cluster Mode?

In Client Mode, the Driver runs on the machine where the application is submitted. The executors run in the cluster, but the driver stays on the client machine. This mode is useful during development and testing because logs are easier to see.

In Cluster Mode, the Driver runs inside the cluster itself. The client only submits the application and does not need to stay connected. This mode is better for production jobs because the application can continue running even if the client machine disconnects.

So, Client Mode is mostly used for testing, while Cluster Mode is preferred for production.

Q14: Write a query to filter a dataset for rows where the region is 'North' OR the priority is 'High'.

```
filtered_df = df.filter(
    (df["region"] == "North") | (df["priority"] == "High")
)

filtered_df.show()
```

Q15: When exploring a dataset, why is it safer to use .show(5) instead of .collect() on a multi-terabyte dataset?

It is safer to use .show(5) because it displays only the first 5 rows of the dataset. It does not bring the full data to the driver.

On the other hand, `.collect()` brings the entire dataset from executors to the driver machine. If the dataset is very large, like multi-terabyte data, the driver memory can get full and the application may crash.

So, for large datasets, `.show(5)` is better for checking sample data, while `.collect()` should be avoided unless the data is very small.